Simplifying the K215 Correctness Proofs

Native Replacement, Shared Runtime Contracts,
and Clock-Free Termination

Technical proposal for Vladimir Reshetnikov's ProveIt repository

Prepared with ChatGPT; source audits and checked component prototypes

22 October 2026

Abstract

The current K215 endpoint is already a small composition of substantial shared theorems. Further simplification should target redundant semantic constructions, duplicated runtime implementations, and the induction interfaces used by their clients. We give three concrete proposals against a pinned repository revision. First, a checked Lean patch derives Replacement from the quotient's existing Collection theorem and separates a generic module from its legacy specializations. It reduces the literal endpoint's static project-source closure by 165 physical lines and 8,358 bytes, while retaining every old interface. Second, an audit identifies 318,724 bytes in seven additional copies of five Rocq runtime modules; a checked finite-support transport prototype specifies a sound reuse boundary, but the common runtime refactor remains to be implemented. Third, a checked Lean adapter replaces clock subtraction in the current simulation theorem with induction over reachable suffixes. Its client is shorter, but the new library has a net source cost for a single client.

The report distinguishes source size, proof architecture, assumption profiles, and completed verification. These are component-level improvements, not a fresh full-machine certification or an internal derivation in ZFC.

Recommended order. Integrate native Replacement and its import split first. Port the existing Lean table-parametric runtime boundary to Rocq next. Adopt the termination library separately when additional clients justify its maintenance cost. The exact 215-state machine, source program, and consistency statement are preserved by the supplied Lean proposals.

Source pin: `cbf94cb552a3b49509e0eee80e7455d4eb5974c6`. All repository references below use this immutable revision. Source counts include comments and blank lines unless explicitly stated otherwise.

Contents

1 The target and the current baseline **3**
1.1 What must be preserved . . . 3
1.2 Fresh source measurements . . . 3
1.3 Substantial simplifications already exist . . . 4

2 Recommendation and expected benefit **4**

3 Native Replacement after quotient Collection **5**
3.1 The existing route repeats quotient work . . . 5
3.2 The exact Collection contract . . . 5
3.3 Formula indexing and variable order . . . 6
3.4 The import split is part of the proof simplification . . . 6
3.5 Measured effect and compatibility . . . 7

4 A shared Rocq runtime for the K family **8**
4.1 The duplication is measurable . . . 8
4.2 Use the boundary Lean already proves . . . 8
4.3 A precise transport theorem for finite table access . . . 9
4.4 A concrete refactoring sequence . . . 10

5 Termination transport without client clock arithmetic **10**
5.1 The theorem . . . 10
5.2 Induction over all reachable suffixes . . . 11
5.3 Connecting the exact current semantics . . . 12
5.4 What actually becomes shorter . . . 12

6 Preserving the machine and the certificate boundaries **12**

7 Verification performed and its limits **13**
7.1 Native Replacement component . . . 13
7.2 Termination component . . . 14
7.3 Rocq runtime prototype . . . 14
7.4 Historical endpoint evidence remains separate . . . 14

8 Adoption plan and acceptance criteria **15**
8.1 First change: native Replacement and its import split . . . 15
8.2 Second change: a common Rocq register runtime . . . 15
8.3 Optional separate change: shared termination induction . . . 15
8.4 Measure the right things . . . 16

9 The internal ZFC objective **16**

A Archive contents and reproduction **16**
A.1 Contents . . . 16
A.2 Requirements and commands . . . 17

B Literal identity and provenance **18**

1 The target and the current baseline

1.1 What must be preserved

For the accepted literal K215 machine M_{215}, the formal endpoint is

$$\text{Halts}(M_{215}, \text{blank}) \quad \longleftrightarrow \quad \text{there is a finite ZFC derivation of falsity.} \tag{1}$$

Negating both sides gives nonhalting if and only if syntactic Con(ZFC). No consistency assumption is a premise of this equivalence. The intended application is a machine whose behavior expresses the consistency question; no simplification here claims unconditional nonhalting or a smaller state count.

The repository identifies a specific dense table: 215 working states, two symbols, one two-way infinite tape, blank input, initial state 0, and a separate halt. Initialization is included in the state count. Both maintained proof lanes target this same table, rather than merely two machines of the same size [1, 2].

There is a distinction relevant to the user's phrase *provably equivalent in ZFC*. The repository's verification statement explicitly describes external Lean and Rocq theorems and says that an internal PA proof or a derivation encoded in ZFC's own proof syntax is not supplied [1]. This report preserves that status. It does not turn an external proof-assistant theorem into an internally certified ZFC derivation by changing its presentation. Section 9 explains the remaining interface.

1.2 Fresh source measurements

The report's read-only census follows imports from `BusyBeaver.ZFC.KEndpoint215`, resolves project modules through the declared source roots in `lakefile.toml`, and excludes the Lean toolchain's `Init`, `Std`, and `Lean` sources. It records the hash and import list of every selected source. These are *syntactic import-closure* measurements; they are not counts of declarations actually referenced in the final proof term.

Pinned-source observation	Value
Project modules in the literal Lean endpoint closure	174
UTF-8 bytes in that closure	1,752,650
Physical lines in that closure	37,648
Physical lines in `KEndpoint215.lean` itself	54
Rocq `coq/215/` files / physical lines	44 / 8,026
Dense machine actions	430
Expanded source leaves	4,751
Guarded site outcomes, counting both decrement cases	6,176
Maximum expanded source depth / checker bound	47 / 48

Table 1: Current-source counts. The machine and source counts are reproduced in `evidence/baseline-census.json`; the Rocq counts are in `rocq-runtime/runtime_audit.json`.

The leaf counts are 1,900 increments, 1,425 decrements, 1,327 continues, 98 breaks, and one halt. The literal bootstrap has 95,714 steps; its representation uses gap 65 and 40 spare register blocks. These are identity and boundary facts, not proposed optimization targets [4, 2, 5].

The repository's phase-15 summary reports an earlier 176-module closure. Subsequent changes are present at the pin used here. Comparing the proposed overlay against that historical figure would mix baselines. The fresh 174-module census is the baseline for every closure reduction in this report.

1.3 Substantial simplifications already exist

The maintained proof already has the principal architecture one might suggest on a first reading: a generic K-family endpoint, positive-progress simulation, machine-specific finite certificates, and a shared logical calibration [3, 7, 6]. In particular:

- The final Lean theorem instantiates a shared composition with dense renaming, register-engine contracts, boot reachability, site coverage, and source correctness.
- The source correspondence uses a normalized 1,426-entry certificate. The reverse direction is recovered from symmetry of the checked bisimulation, rather than proved by a separate equally large certificate.
- The current lower semantic chain constructs the seed property on the original model and shares quotient syntax and recursion infrastructure.
- The source tree is already computed from the source program. Replacing the former large literal tree by that definition is not a new opportunity at this pin.
- Rocq's final source and correctness files are already short facades. Recent commits have also removed unused initial-state enumerator theorems and shared paired dispatch tactic scripts.

Accordingly, the proposals below concentrate on changes that remain useful at the current revision. The guarded-image and reachable-suffix ideas also occur in the repository's earlier proposal packets. The contribution here is current-source integration, exact dependency accounting, explicit obligations, and fresh component checks, not priority for those ideas [12, 13, 19, 18].

2 Recommendation and expected benefit

Change	Completed result	Benefit and limit
Native Replacement	Current Lean overlay, compatibility clients, generic endpoint compilation	Active closure: −165 lines, −8,358 bytes. All maintained affected sources: +84 lines.
Shared Rocq runtime	Eight-variant census; kernel-checked adaptive-query transport and K215/K217 support witness	318,724 bytes in seven extra copies of five modules. This is a duplication inventory, not an achieved net saving.
Termination transport	Generic Lean theorem, exact adapters, actual `KSim` patch, checked counterexamples	Client file: −28 lines. Client plus new library: +188 lines. Best treated as a shared abstraction.

Table 2: Three separately evaluated changes. Their metrics must not be added as though a combined full endpoint build had been performed.

The semantic change is the first adoption candidate because it removes an entire repeated style of reasoning from the active proof: representative selection, bisimulation invariance, and quotient descent for Replacement. The Rocq consolidation is the larger maintenance opportunity because changing a shared runtime theorem should require changing one proof. The termination abstraction improves local reasoning, but its source cost makes it a secondary recommendation.

The common principle is to prove a semantic property at the layer where its natural equality and invariants are already available. It is usually more valuable to remove a second proof route than to shorten individual tactic scripts within both routes.

3 Native Replacement after quotient Collection

3.1 The existing route repeats quotient work

`KQ1Collect.quotient_collection` already supplies Collection for every actual first-order formula, with arbitrary parameter valuations in the quotient model. Its proof has already handled representative lifting and the equivalence relation on raw objects [9].

The current `KQ1Replacement` independently introduces a guarded raw matrix, proves its invariance, obtains interpreted Replacement, and descends the resulting image to the quotient [10]. That is a valid route, and its public helpers may remain useful. But the active native model already has the interface needed for a much shorter argument: Collection with literal equality.

The proposed route is

$$\text{quotient Collection} \implies \text{guarded functional image} \implies \text{native Replacement}.$$

The new production file is only 74 physical lines, including the formula adapter and both quotient wrappers. The elementary image lemma itself is independent of the set-theoretic construction.

3.2 The exact Collection contract

It is essential to use the repository's actual contract [8]. For a membership relation m, write $x \in a$ for $m(x, a)$. For a binary relation r, the proposition ImageCollection(r, a, b) is

$$\forall y \in b\ \exists x\ r(x, y), \tag{2}$$

$$\forall x \in a\ ((\exists y\ r(x, y)) \implies \exists y \in b\ r(x, y)). \tag{3}$$

Collection for r means that such a b exists for every a. The support clause (2) does *not* restrict its witness x to the particular domain a. This makes the domain guard necessary. An unguarded application followed by functionality does not prove exact image membership.

Lemma 3.1 (Guarded functional image)**.** *Let $r_a(x, y)$ abbreviate $x \in a \wedge r(x, y)$. Suppose Collection holds for r_a, and suppose r is partially functional on a:*

$$x \in a \wedge r(x, y) \wedge r(x, z) \implies y = z.$$

Then there is a b such that

$$y \in b \quad \longleftrightarrow \quad \exists x\ (x \in a \wedge r(x, y)).$$

Proof. Apply Collection for r_a at domain a, obtaining b. If $y \in b$, the support clause gives x with $r_a(x, y)$, hence $x \in a \wedge r(x, y)$.

Conversely, let $x \in a$ and $r(x, y)$. Coverage for r_a supplies $z \in b$ with $x \in a \wedge r(x, z)$. Functionality gives $z = y$, so $y \in b$. □

This proof requires neither totality of r on a, nor extensionality, nor an inhabited carrier, nor a choice principle. The checked Lean `exact_image` theorem reports no axioms. Partial functionality is enough: inputs with no output make no contribution to the image.

The finite examples in `scripts/check_guard_examples.py` illustrate the necessity of the hypotheses. With $a = \{0\}$, $r = \{(0, 0), (1, 1)\}$, the unguarded collection $b = \{0, 1\}$ satisfies global support and coverage, but has an unwanted member. Without functionality, $r = \{(0, 0), (0, 1)\}$ permits the covering set $\{0\}$, which misses an output. Without support, coverage permits extra members. These finite examples explain the interface; the general result is proved in Lean, not inferred from the examples.

3.3 Formula indexing and variable order

The model supplies Collection for formulas, not arbitrary semantic predicates. The generic mathematical relation r_a must therefore be realized by an actual formula. The two existing APIs use different variable conventions:

API	Slots 0 and 1	Remaining slots
`relOf phi env y x`	output y, input x	original parameters
`MatrixRelation`	input x, output y	collection parameters
Guarded matrix	input x, output y	domain a, then parameters

Define a renaming $\rho : \mathbb{N} \to \mathbb{N}$ by

$$\rho(0) = 1, \qquad \rho(1) = 0, \qquad \rho(k+2) = k+3.$$

The concrete guard is

$$\mathrm{guardForm}(\varphi) := \mathrm{Mem}(0,2) \wedge \mathrm{rename}(\rho, \varphi).$$

In the environment (x, y, a, env), its semantics is exactly

$$x \in a \ \wedge \ \mathrm{relOf}(m, \varphi, \mathrm{env}, y, x).$$

The supplied `sat_guardForm` proves this identity using the maintained renaming theorem. Applying Collection to this formula and to the parameter environment (a, env) supplies the premise of Lemma 3.1. The result is then converted by the existing `bridge_Repl` to satisfaction of the usual Replacement schema.

This adapter does not introduce Collection for every semantic predicate. Every call uses one concrete `Form`. The original `QHyp` assumptions are unchanged. There is no second equality-free translation: the quotient Collection theorem already interprets native equality.

3.4 The import split is part of the proof simplification

Replacing only `KQ1ZF`'s direct call is insufficient. The old development remains in its transitive imports through

$$\texttt{KQ1ZF} \longrightarrow \texttt{InductiveAttempts} \longrightarrow \texttt{QuotientReplacement} \longrightarrow \texttt{KQ1Replacement}.$$

The intermediate finite-attempt module contains a generic argument followed by CP/GIC specializations. Only the specializations require the old quotient Replacement interface [11].

The patch separates these parts at their existing boundary:

`InductiveAttemptsCore.lean`
The generic finite-attempt development, with its original proof bodies unchanged. It has no dependency on the legacy Replacement specialization.

`InductiveAttempts.lean`
A compatibility facade importing the core and retaining the original specialization tail. Old declarations and module imports remain available.

`KQ1ZF.lean`
Imports the core and the new native Replacement module. Its Replacement field and its call to `ordinary_infinity_of_container` receive the new native theorem.

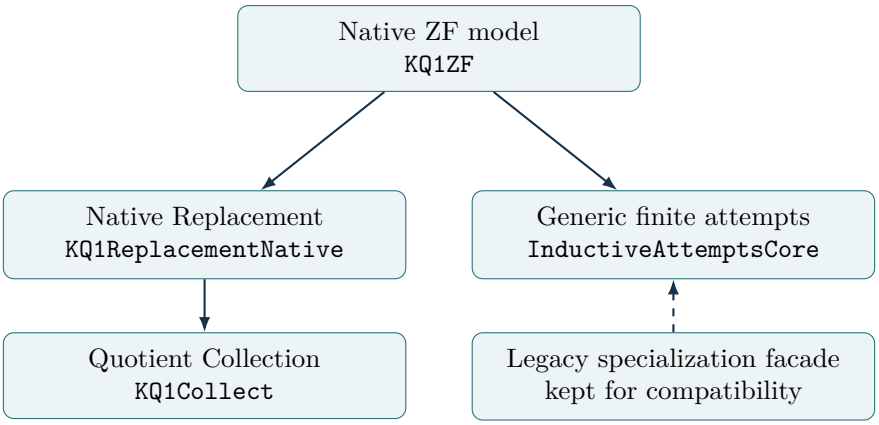


Figure 1: Proposed dependency boundary. Arrows indicate imports or use of the corresponding proof interface. The compatibility facade imports the core; the active model does not import the facade.

The source-split audit checks byte-for-byte preservation of both the generic and specialization proof bodies, and confirms that the old `KQ1Replacement` and `QuotientReplacement` files are unchanged. This is a modest structural edit, rather than a rewrite of the infinity proof.

The change is intentionally limited to Replacement. Existing collection infrastructure uses separation-related valuation lifting, and the power construction still needs raw subdomain reasoning. Moving every construction to the native model at once would risk a dependency cycle or merely recreate an auxiliary raw construction under a new name. The supplied patch makes no such additional change.

3.5 Measured effect and compatibility

Literal endpoint source closure	Baseline	Proposed	Difference
Project modules	174	173	−1
UTF-8 source bytes	1,752,650	1,744,292	−8,358
Physical lines	37,648	37,483	−165

Table 3: Native Replacement plus the necessary import split. These are independently reproduced static import counts.

The active closure loses `InductiveAttempts`, `QuotientReplacement`, and `KQ1Replacement`, and gains `InductiveAttemptsCore` and `KQ1ReplacementNative`. Changing only the direct Replacement call would instead add one module, 3,544 bytes, and 74 lines to the active closure, because the old chain would remain imported.

Compatibility has a real cost. Across all affected maintained production files, including the retained facade and old proofs, this patch adds 3,902 bytes and 84 physical lines. It therefore simplifies and shrinks the active endpoint closure; it does not reduce total repository source. Legacy helper deletion could be considered after auditing all clients, but no such deletion or additional saving is assumed here.

The old nine-declaration Replacement API and the finite-attempt specializations are checked by a separate audit client. The two old and new Replacement endpoints have identical proposition types. An audit equality proved by `rfl` uses Lean proof irrelevance: it verifies type compatibility, not identical proof algorithms, proof-term sizes, or elaboration cost.

4 A shared Rocq runtime for the K family

4.1 The duplication is measurable

The Rocq extension currently keeps physically separate variant directories for K215, K217, K218, K221, K225, K226, K227, and K229. The supplied audit removes nested comments, normalizes only namespace tokens of the form `K<digits>F`, and removes blank lines and line-edge whitespace. Under exactly this normalization, each of the following modules has one common body across all eight variants. These bodies import different candidate constants. They cannot simply be

Module	Eight copies	K215 copy	Other seven
`Zip`	30,923	3,959	26,964
`Runtime230Core`	156,227	19,622	136,605
`Tape230Blocks`	69,539	8,786	60,753
`Tape230Carry`	43,883	5,579	38,304
`Interpreter230Sites`	64,219	8,121	56,098
Total bytes	364,791	46,067	318,724

Table 4: Physical UTF-8 bytes in the pinned sources, after establishing normalized body equality. The byte totals themselves include original comments and formatting.

deduplicated by textual deletion. The proposed common library must expose the table and its hypotheses as parameters, then supply variant adapters. The inventory is consequently an indication of repeated material, not a measured net saving after refactoring. A single machine will still depend on the common runtime proof.

Several numerical-calculus files also normalize identically between K215 and K217. Their generator explicitly copies them with namespace changes. That is a further consolidation opportunity, separate from the measured five-module runtime target [16]. It should follow a small successful runtime extraction rather than enlarge the first change.

4.2 Use the boundary Lean already proves

Lean's `KTape` already states a useful table-parametric contract [14]. Its register engine uses the 23 sparse state labels

$$0, 1, 2, 3, 16, 22, 25, 32, 33, 36, 40, 48, 51, 54, 55, 60, 64, 66,$$
$$74, 94, 97, 113, 198.$$

Both read-symbol rows are included except the read-0 rows of 113 and 198: 44 rows in total. A separate `IncTail` hypothesis accepts either the original three-row return path or the bypass

$$(198, 0) \longmapsto (1, L, 40).$$

From agreement on the 44 rows and the tail hypothesis, `KTape.boxLemmas` proves all four register-oracle contracts: mark, increment, zero decrement, and positive decrement.

The quantifiers are essential. These are theorems for all valid register files, all required gaps and register positions, and arbitrary left tape frames. They are not checks of selected small register values. Rocq's `Runtime230Core` already lists the corresponding contracts as Section hypotheses [15]; the missing architectural step is to make their implementation and its clients independent of each concrete table.

The pinned sparse tables give the following comparison:

Compared with K215	Differing rows overall	Differences on 44 engine rows
K217	55	0
K218	118	0
K221	129	0
K225	157	0
K226	177	0
K227	181	0
K229	193	0

K215, K217, and K218 use the bypass; the larger listed variants use the original tail. These comparisons use common sparse labels, not candidate dense state numbers. A missing transition is represented as none and compared as a value. In particular, row $(183, 0)$ is missing in K215 and present in K217, so the tables are not globally equal.

4.3 A precise transport theorem for finite table access

The supplied `OracleFootprint.v` provides a small checked interface for reusing table-dependent computations. It is independent of the repository's runtime code and imports only the Coq standard library.

Let Q be query keys, A answers, and R results. A terminating adaptive query computation has the form

$$P ::= \mathrm{Done}(r) \mid \mathrm{Ask}(q, k), \qquad k : A \to P.$$

A table $T : Q \to A$ determines its evaluation:

$$\begin{aligned} \mathrm{eval}_T(\mathrm{Done}(r)) &= r, \\ \mathrm{eval}_T(\mathrm{Ask}(q, k)) &= \mathrm{eval}_T(k(T(q))). \end{aligned}$$

The corresponding query list is empty at Done, and q followed by the query list of $k(T(q))$ at Ask.

Theorem 4.1 (Adaptive footprint transport)**.** *For the same query tree P, suppose T and U agree on every key in* $\mathrm{reads}_T(P)$*. Then*

$$\mathrm{eval}_U(P) = \mathrm{eval}_T(P) \quad \textit{and} \quad \mathrm{reads}_U(P) = \mathrm{reads}_T(P).$$

Proof. Proceed by structural induction on P. The result is immediate for Done. For $\mathrm{Ask}(q, k)$, agreement includes the first key q, so $T(q) = U(q)$. Both evaluations therefore select the same continuation. The remaining agreement hypothesis is exactly the one needed by the induction hypothesis on that continuation. □

Answers may be optional transitions. The prototype's specialized result is $\mathrm{option}(C \times \mathrm{bool})$, so the theorem preserves failure, the exact resulting configuration, and the progress bit. It does not merely preserve a Boolean acceptance verdict.

The supplied `Support215217.v` includes the two complete sparse tables, checks agreement on the 44 engine rows plus the shared bypass, and proves that the tables differ globally. Its negative control rejects the missing $(183, 0)$ row. Kernel evaluation discharges the finite checks; Python is used only to produce the literal data and audit its provenance.

The theorem's *same-tree* premise must remain visible. To apply it to the maintained symbolic executor, prove that execution is evaluation of a single table-independent query tree, or give an equivalent direct simulation theorem for that executor. A generator that chooses the tree

after consulting the baseline table does not establish this premise by itself. That reification or representation theorem is not supplied here.

Likewise, the supplied `supported_computation_reuse` theorem is conditional on a proof that every query lies in the certified support. Agreement on 45 rows does not establish equivalence of whole K215 and K217 machines, does not cover arbitrary source sites, and does not replace the universally quantified register-oracle proofs.

4.4 A concrete refactoring sequence

The common runtime should be built in small layers:

1. **Separate state and tape types from the table.** The sparse K-family variants share the state bound 230. Parameterize the transition table while retaining the existing bounded-state context, zipper, and lifting facts.
2. **Separate representation from boot.** Move common tape-shape definitions and arithmetic lemmas out of the candidate-specific boot module. Supply the blank-to-dispatch reachability proof as a candidate obligation.
3. **Port the register-engine contract.** Prove the Rocq counterpart of the following implication, with its full quantification:

   $$\mathrm{AgreeRows} \wedge \mathrm{IncTail} \implies \mathrm{BoxLemmas}\,.$$

   This gives the main runtime proof a compact, explicit assumption boundary.
4. **Make symbolic execution table-parametric.** Retain the current `oracle_sound`, `sstep_sound`, and `exec_sound` arguments once, with the exact return type and progress bit. Use footprint transport only after proving the executor representation.
5. **Make decoder depth explicit.** Parameterize the program and its maximum decoding depth. Keep the depth bound and source well-formedness in the candidate record.
6. **Instantiate a common simulation theorem.** Each candidate supplies the table, source, boot registers, boot reachability, terminal-row property, coverage, and source correspondence. Keep the existing public theorem names as short wrappers.
7. **Regenerate data and certificates only.** Remove generator substitutions in proof bodies after the adapters and final dense endpoint pass the production toolchain checks.

This addresses a concrete fragility. The current generator changes namespaces, disabled-row guards, boot constants, source names, register-file names, and decoder bounds by string substitution. The repository records a repair for missed comparisons against an old bound of 40 when constructing the bound-48 variant [16, 17]. Explicit parameters make such dependencies part of theorem types.

Home acceleration and register readiness must retain their current hypotheses. A footprint certificate concerns table accesses; it cannot establish the semantic precondition for taking an accelerated macro. Similarly, source coverage, literal terminal behavior, and the boot representation remain separate obligations.

5 Termination transport without client clock arithmetic

5.1 The theorem

Let $f : A \to \mathrm{Option}(A)$ and $g : B \to \mathrm{Option}(B)$ be deterministic partial transition functions. A stopped configuration has value none and no outgoing step. Write $a \to_f^* a'$ for finite reachability, $a \to_f^+ a'$ for nonempty reachability, and $\mathrm{Halts}(f, a)$ for reaching a stopped state.

Theorem 5.1 (Termination transport)**.** *Let $R \subseteq A \times B$. Assume positive forward simulation:*

$$R(a,b) \wedge f(a) = \mathrm{some}(a') \implies \exists b'\ (b \to_g^+ b' \wedge R(a',b')). \tag{4}$$

Then $R(a,b) \wedge \mathrm{Halts}(g,b)$ implies $\mathrm{Halts}(f,a)$. If, additionally, source terminals drain to target termination,

$$R(a,b) \wedge f(a) = \mathrm{none} \implies \mathrm{Halts}(g,b), \tag{5}$$

then

$$R(a,b) \implies (\mathrm{Halts}(f,a) \longleftrightarrow \mathrm{Halts}(g,b)).$$

This is the positive-simulation principle already used in the repository. The contribution is a reusable induction interface that removes the need for the current `KSim.halts_iff` client to compare a macro length with a remaining halting time and subtract the former from the latter [7, 18, 19].

5.2 Induction over all reachable suffixes

Lemma 5.2 (Reachable-suffix induction)**.** *Suppose $P : A \to \mathrm{Prop}$ satisfies*

$$f(a) = \mathrm{none} \implies P(a),$$
$$f(a) = \mathrm{some}(a_1) \ \wedge\ (\forall c\ (a_1 \to_f^* c \Rightarrow P(c))) \implies P(a).$$

If $\mathrm{Halts}(f,a)$, then $P(c)$ for every c reachable from a; in particular, $P(a)$.

Proof. Induct on the length of a finite run from a to a stopped state, strengthening the conclusion to all reachable c. At length zero, a is stopped. The only reachable state is a, because a stopped state has no successor. At positive length, let the first successor be a_1. The induction hypothesis establishes P at every state reachable from a_1. For a reachability witness from a, either its length is zero, in which case use the step premise, or it begins with the same successor a_1, in which case use the induction hypothesis. Determinism identifies the two first successors. □

Proof of Theorem 5.1. For reflection, apply Lemma 5.2 to the terminating target, with

$$P(b) := \forall a\ (R(a,b) \Rightarrow \mathrm{Halts}(f,a)).$$

If b is stopped and a had a successor, positive simulation would produce a nonempty run from b, a contradiction. Hence a is stopped.

If $g(b) = \mathrm{some}(b_1)$, consider a related a. When a is stopped there is nothing to prove. Otherwise let its successor be a', and obtain a positive target macro ending at b'. Remove the macro's first step. Its remaining suffix witnesses $b_1 \to_g^* b'$, so the strengthened induction hypothesis applies at b' and gives $\mathrm{Halts}(f,a')$. Prepending the source step proves $\mathrm{Halts}(f,a)$.

For preservation, induct on a terminating source run. The terminal case uses (5); a source step uses the macro from (4) followed by the target termination supplied at its endpoint. □

No bound on macro length is needed. The relation need not be functional or injective, and need not describe internal configurations of a target macro. The proof chooses one macro only at the current induction step; it does not construct a global infinite choice sequence. The checked generic suffix, reflection, and equivalence theorems report no axioms.

5.3 Connecting the exact current semantics

The supplied `TerminationTransportK` proves that the generic finite-run function agrees with the existing `KZip.run` and `RegisterPath.run`. It also converts their halting predicates. The actual `KSim` patch then supplies three existing facts: positive simulation from `sim_step`, terminal behavior from `sim_halt`, and source completeness from `sim_stop hcov`.

The last fact is indispensable. In the source interpreter, `RegisterPath.step = none` can arise from decoder exhaustion as well as a literal halt instruction. The adapter therefore explicitly requires

$$\texttt{step p c = none} \implies \texttt{AtHalt p c}.$$

The maintained coverage certificate supplies this premise. Replacing literal source halting by an arbitrary partial-function deadlock without it would change the theorem.

Two checked Lean counterexamples document the other boundaries:

- A one-state source loop and an immediately stopped target satisfy a zero-or-more-step version of simulation. Their halting behavior differs. Strict progress in (4) cannot be dropped.
- A stopped source and an infinite target satisfy all source-transition obligations vacuously. Preservation fails without (5).

For nondeterministic targets, an existential forward simulation may follow a loop even when another target branch halts. That semantics requires a different theorem. An absorbing totalized halt also requires an adapter that removes outgoing steps after halting; otherwise positive progress could occur after the halt. Existing proofs that address absorption must retain those hypotheses.

5.4 What actually becomes shorter

Physical lines	Original	Proposed
Entire `KSim` file	354	326
`halts_iff`, declaration and proof	35	6
New generic termination library	0	144
New exact K-family adapters	0	72
Production files combined	354	542

The client file shrinks by 28 lines, but this isolated extraction adds 188 production-source lines. The counterexamples and replay code are excluded from that comparison. This is useful when multiple clients share the rule, or when removing fragile local clock arithmetic justifies the library cost. It is not the main source-size result.

The adapter also proves a direct replacement for `KEndpoint.halts_of_reach`. That optional endpoint edit is not in the patch and was not checked through a rebuilt endpoint. Other clients need their own semantics adapters. In particular, the internal arithmetic-model theorem discussed in Section 9 is not an automatic client.

6 Preserving the machine and the certificate boundaries

The exact transition table is part of the theorem, not merely an implementation detail. The supplied changes preserve the source and table definitions. For later Rocq consolidation, a candidate interface should make the following obligations independently visible.

Boundary	Required statement
Dense versus sparse states	The dense table is the specified sparse table under the explicit state renaming, including initial and terminal conventions.
Register engine	All four oracle contracts hold under the full tape and register invariants; finite row agreement is used only through a proved sufficient-support theorem.
Boot	The literal blank configuration reaches the dispatch configuration with the specified source path, registers, gap, and spare blocks.
Decoder and site coverage	Every actual leaf, including both decrement outcomes, is covered within the declared depth bound. The checker rejects unsupported or exhausted decoding.
Positive simulation	Each nonterminal source step has a nonempty target execution to a related configuration. Accelerated macros retain their readiness conditions.
Source correspondence	The actual source program and its initial registers have the stated equivalence with the proof enumerator.
Logical calibration	The enumerator derives falsity exactly when ZFC has a finite refutation, through the existing numerical-calculus and ZF/ZFC calibration.

Lean already exposes most of this structure in its generic endpoint. The proposal is to preserve it there and reproduce the useful boundary in Rocq, not to replace all these obligations by a stronger unchecked *candidate is correct* record.

For the site certificates, the appropriate further research question is whether a shared decoder-prefix evaluator and common macro contracts reduce repeated proof computation across the 6,176 guarded outcomes. The repository already contains work on decoder prefixes and related certificate factorizations [6]. A new claim of saving must therefore be measured against those current certificates, preserve failure and progress information, and include a rebuilt coverage theorem. The present packet does not claim that additional optimization.

An untrusted generator remains a sensible architecture. It can emit tables, source syntax, certificates, and proposed footprints. The proof assistant checks their finite predicates using proved soundness theorems. A Python comparison or a hash binds provenance and detects accidental changes; it does not prove the semantic soundness of a generated certificate.

7 Verification performed and its limits

7.1 Native Replacement component

Lean 4.32.0 freshly compiled 107 project modules and two audit modules from fresh project objects. The build includes the changed lower semantic chain, the unchanged generic `KEndpoint.blank_halts_iff_zfc_refutation`, and the legacy Replacement and finite-attempt clients. All 109 compiler exits were zero. The audit clients emitted no warnings or errors; the dependency modules emitted 15 existing warnings. No `sorryAx` occurs in the recorded assumption reports.

Declaration	Reported assumptions
`exact_image`	None
`sat_guardForm`, `replacement_of_collection`	`propext`
New and old quotient Replacement endpoints	`propext`, `Classical.choice`, `Quot.sound`
`KQ1ZF.zf_consistent`, generic `KEndpoint`	`propext`, `Classical.choice`, `Quot.sound`

The 109 compiler processes took 55.58655 seconds in this environment. There is no paired timing experiment, so this number is a reproduction observation, not evidence of a speedup. The patch passes `git apply -check` at the pinned revision.

The concrete K215 source, boot, and site certificate closure was not recompiled under the overlay. Static counts of that closure do not imply a fresh full-machine build. The generic endpoint check is substantial evidence for the semantic change, but the final literal endpoint remains the integration gate.

7.2 Termination component

Lean 4.32.0 rebuilt all 13 original runtime modules, then checked the new generic library, exact adapters, counterexamples, and actual patched `KSim`, reusing the 12 unchanged dependency objects from that fresh baseline.

Both original and changed `KSim.halts_iff` and `KSim.sim_step` report exactly

`propext`, `Classical.choice`, `Quot.sound`.

The generic transport theorems and the two counterexamples report no axioms. The full K215 endpoint was not rebuilt in this separate experiment, and the two Lean changes were not tested as one combined overlay.

7.3 Rocq runtime prototype

The prototype was compiled with **Coq 8.16.1, OCaml 4.13.1**. This differs from the repository's production **Rocq 9.1.0 with rocq-stdlib 9.0.0**. Both prototype files compiled, and all six requested assumption reports say *Closed under the global context.*

The command

```
coqchk -silent -o OracleFootprint Support215217
```

recursively rechecked the modules and their standard-library dependencies. It reported no axioms, type-in-type, unsafe fixpoints, or assumed positivity. The checker took approximately 24.20 seconds in this environment. No `-admit` or `-norec` option was used.

This is a checked transport lemma and finite literal-table witness. It is not a port of the register-engine proof, an adapter for the maintained symbolic executor, or a production-toolchain build of K215.

7.4 Historical endpoint evidence remains separate

The repository records earlier full-machine builds at revision `613f9cf28a`: 204 freshly compiled Lean project modules and two endpoint clients, and 237 Rocq sources with recursive checking.

The Lean endpoint assumptions are the three listed above; the Rocq endpoint dependency reports are closed [1].

Those historical records are not relabeled as checks of this packet. The supplied receipts bind the new component inputs, commands, source snapshots, outputs, and logs. An independent static review within this collaboration reproduced the endpoint census and checked source and log hashes; it is not an external human audit.

8 Adoption plan and acceptance criteria

8.1 First change: native Replacement and its import split

Review the four production files and the split audit together. Apply the native patch against the pinned source, replay the 109-module component check, and then run the maintained full K215 endpoint driver with freshly rebuilt affected objects. Print the literal endpoint's assumptions. Retain the machine/source identity checks and all compatibility clients selected by the repository's integration policy.

The acceptance criteria are:

- Identical public consistency statements and machine/source definitions.
- The legacy API client passes, and the original finite-attempt proof bodies remain unchanged.
- The final literal endpoint compiles with the expected three Lean assumptions and without a new admitted proof.
- The import census reproduces the measured reduction, with total maintained-source growth reported alongside it.

The raw Replacement module can remain as a compatibility implementation. If later client analysis permits removing it, that should be a separate change with its own measurements.

8.2 Second change: a common Rocq register runtime

Start with K215 and K217, since they share the bypass and their 44 engine rows agree. Extract the common table/tape interface, prove the universal register contract, and instantiate both adapters. Only then move the symbolic executor and its soundness proofs. Finally parameterize the program, depth, coverage, and boot inputs.

Run the production Rocq toolchain and recursive checker at each boundary. Require the same final theorem types and endpoint assumption reports. Keep candidate wrappers stable so that source and dense-table clients do not need to follow the implementation's module reorganization.

The copied proof files should be removed only after the common proof and all intended variant adapters pass. The resulting measurement should separately report common-library size, adapter size, total maintained source, and the transitive closure of one representative endpoint. The 318,724-byte inventory is a useful target for consolidation, not an acceptance criterion that new adapter code must somehow avoid counting.

8.3 Optional separate change: shared termination induction

The supplied KSim patch is already checked locally. Adopt it as a separate architectural change, so its new imports and cost remain reviewable. A second suitable external partial-machine client can use the same suffix induction rule. Do not attribute hypothetical savings from unported clients to this change.

8.4 Measure the right things

There are at least four different meanings of a *shorter proof*:

1. Less maintained source across all variants and compatibility layers.
2. A smaller import closure for the theorem of interest.
3. A smaller or faster-to-check elaborated proof object.
4. Fewer distinct mathematical constructions or invariants for a reader to understand.

This packet measures the first two, supplies evidence about assumption profiles, and explains the fourth. It does not measure proof-object size or comparative build performance. A later performance claim should use the same compiler, source pin, object-cache policy, and requested target set, with repeated runs and reported variability.

9 The internal ZFC objective

For the stated mathematical goal, the desired internal statement can be written schematically as

$$\mathsf{ZFC} \vdash \left(\neg\,\mathrm{Halt}_{\ulcorner M_{215} \urcorner} \leftrightarrow \mathrm{Con}(\mathsf{ZFC})\right).$$

Here the machine code, halting predicate, proof predicate, and formal syntax all need specified encodings. The external formal equivalence is valuable evidence and a substantial formal theorem. An internally encoded derivation is a further artifact with its own proof obligations.

This distinction suggests a useful constraint on simplification: keep finitary machine, decoder, and simulation interfaces explicit, and keep the logical calibration theorem's strength visible. A short external proof that depends on a stronger semantic existence theorem is not automatically a shorter internal proof of the intended statement.

In particular, `ArithmeticPositiveSimulation254` proves a theorem inside arbitrary arithmetic models using definability and induction conditions. The external Lean suffix-induction theorem does not replace those internal conditions. It may guide the shape of an internal proof, but using it directly would discard the very interpretation guarantee the arithmetic theorem is designed to establish [20].

A future internalization effort should therefore expose and check: the exact coded machine and proof system; the representability of each finite transformation; the induction principles used in simulation and enumeration; and the internal form of the ZF/ZFC consistency calibration. No new internal PA or ZFC endpoint is claimed by this report. The proposed refactors aim to make those boundaries easier to inspect while preserving the currently checked external theorem.

A Archive contents and reproduction

A.1 Contents

Path	Purpose
`report.tex`, `report.pdf`	This complete report and its compiled PDF.
`README.md`, `Makefile`	Entry point and documented reproduction commands.
`lean-native/`	Four-file production overlay, repository-relative patch, two audit clients, replay driver, source census, split audit, and final component receipts, logs, and source snapshots.

Path	Purpose
`lean-termination/`	Generic theorem, exact adapters, counterexamples, full KSim overlay and patch, pinned source hashes, replay driver, and evidence.
`rocq-runtime/`	Generic adaptive-query proof, complete K215/K217 sparse-table witness, generator/audit, compiler driver, and recursive-check logs.
`scripts/`	Read-only source census, finite explanatory checks, and bundle integrity verification.
`evidence/`	Root census, finite-check receipt, mathematical/source reviews, and packaging verification.
`LICENSE`, `NOTICE.md`, `licenses/`	Project license and retained machine/source attribution and inherited terms.
`MANIFEST.json`	SHA-256 and byte count of each distributed file, excluding the manifest itself.

No compiler binaries, compiled proof objects, or repository clone are included. Original dependency sources in the native component's receipt snapshots are retained to make the accepted compiler inputs inspectable. Absolute paths in historical receipts describe the environment of the recorded run; replay drivers take new explicit paths.

A.2 Requirements and commands

Use Python 3.11 or later for the audit scripts, Lean 4.32.0 for the Lean components, and Coq 8.16.1 to reproduce the recorded Rocq-prototype run. Production integration must use the repository's pinned Rocq toolchain. The Lean replay drivers need the materialized project source roots from the pinned repository; they do not need a prebuilt project cache or Mathlib.

From an extracted archive directory:

```
python3 scripts/verify_bundle.py

python3 scripts/census.py \
  --repo /absolute/path/to/ProveIt \
  --out /tmp/k215-baseline-census.json

python3 lean-native/replay.py \
  --repo /absolute/path/to/ProveIt \
  --lean /absolute/path/to/lean-4.32.0/bin/lean \
  --build /tmp/k215-native-fresh

python3 lean-termination/replay.py \
  --repo /absolute/path/to/ProveIt \
  --lean /absolute/path/to/lean-4.32.0/bin/lean \
  --out /tmp/k215-termination-fresh
```

Both Lean output directories must be new. The native replay compiles the generic endpoint and compatibility scope; the termination replay compiles the original and changed runtime scopes. Neither command claims to run the full literal K215 integration gate.

The Rocq producer and checker write their output beside the scripts. Copy the component to a fresh work directory first if preserving the accepted receipts:

```
cp -R rocq-runtime /tmp/k215-rocq-fresh
python3 /tmp/k215-rocq-fresh/audit_runtime.py \
  --repo /absolute/path/to/ProveIt
python3 /tmp/k215-rocq-fresh/check.py \
  --coqc /absolute/path/to/coqc \
  --coqchk /absolute/path/to/coqchk
```

The read-only Python census and the finite explanatory checks are not substitutes for proof-assistant compilation. To rebuild the PDF, run `make report`; it needs a LaTeX installation with the packages listed in the preamble.

B Literal identity and provenance

The independently computed identities of the two principal K215 inputs are:

Dense transition table: `candidate-k215/k215.tm`

```
fdbcef0825886035c3091c8537a69535
bc96f9e265f0ae45863543d40ae70090
```

Expanded source: `candidate-k215/sites215.json`

```
17af207195179b2508bf91154cd00739
374f8862207ca21b9a3cb9098aa6f5ba
```

Each displayed SHA-256 is split across two lines only for typesetting. The JSON receipts contain the unbroken values.

The finite image check enumerates all 512 binary relations and all choices of domain and proposed image on a three-element carrier: 32,768 triples. Exactly 1,728 satisfy the guarded-support, coverage, and functionality premises; all satisfy exact image equality. This is a sanity check of the stated contract, not a finite model of ZFC and not the proof of Lemma 3.1.

The machine, numerical calculus, compiler ancestry, and previous proof developments remain credited to their existing authors. The archive retains the K215 and K217 notices and inherited license materials. Earlier `collection-first`, `native-replacement`, `proof-core`, and `further-interface` proposals are explicitly credited for the ideas implemented here. New packaging and code do not assert discovery priority or relicense inherited third-party work.

References

[1] ProveIt, Formal correctness of the exact 215-state machine. Historical full-build scope, endpoint statements, machine identity, and the external-versus-internal qualification.

[2] ProveIt, `KEndpoint215.lean`. Literal dense-machine endpoint and machine-specific inputs.

[3] ProveIt, `KEndpoint.lean`. Generic machine endpoint and numerical-calculus calibration.

[4] ProveIt, `KBoot215.lean`. Literal bootstrap certificate.

[5] ProveIt, `candidate-k215/sites215.json`, and `KSites215.lean`. Expanded source and machine site coverage.

[6] ProveIt, K215 simplification history. Existing stages, retained evidence, and prior proposal packets.

[7] ProveIt, `KSim.lean`. Positive machine/source simulation, source completeness, and halting equivalence.

[8] ProveIt, `ProfileCollection.lean`. The actual image-support and coverage contract.

[9] ProveIt, `KQ1Collect.lean`. Collection for formula-defined relations on the native quotient.

[10] ProveIt, `KQ1Replacement.lean`, and `KQ1ZF.lean`. Existing Replacement route and the model assembly.

[11] ProveIt, `InductiveAttempts.lean`, and `QuotientReplacement.lean`. Generic finite attempts and the legacy specialization dependency.

[12] ProveIt, Native Replacement proposal. Earlier guarded-image argument, formula adapter, and import-dependency warning.

[13] ProveIt, Collection-first proof-simplification proposal. Earlier native image and collection-first architecture.

[14] ProveIt, `KTape.lean`. The 44-row agreement condition, increment-tail alternatives, and universal register-engine theorem.

[15] ProveIt, `Runtime230Core.v`. Rocq oracle contracts, symbolic execution, and soundness.

[16] ProveIt, `regenerate215.py`. Current variant generation and proof-body substitutions.

[17] ProveIt, Rocq extension documentation. Production proof lane and decoder-bound repair history.

[18] ProveIt, Proof-core proposal. Earlier positive-simulation extraction and related architecture.

[19] ProveIt, Further-interface proposal. Earlier reachable-suffix induction idea.

[20] ProveIt, `ArithmeticPositiveSimulation254.lean`. Positive simulation inside arithmetic models and its definability and induction hypotheses.